物流・運送業BPO詳細設計

スコープ: 一般貨物自動車運送事業者（10-300名規模、車両10-200台）
攻略優先度: A（コア6業界）
ゲノム系統: 建設系（許認可+安全書類+計画の属人化の同構造）
実装状況: #1 配車計画AI パイプライン実装済み（ `workers/bpo/logistics/pipelines/dispatch_pipeline.py` ）
制約: GPSリアルタイム連携はPhase 2+。Phase 1はバッチ配車計画に集中

マイクロエージェント対応表

設計書での呼称	実装関数名	ファイル
OCR/テキスト抽出	run_document_ocr	workers/micro/ocr.py
構造化抽出	run_structured_extractor	workers/micro/extractor.py
ルール照合	run_rule_matcher	workers/micro/rule_matcher.py
コスト計算	run_cost_calculator	workers/micro/calculator.py
コンプライアンスチェック	run_compliance_checker	workers/micro/compliance.py
文書生成	run_document_generator	workers/micro/generator.py
バリデーション	run_output_validator	workers/micro/validator.py
SaaS読取	run_saas_reader	workers/micro/saas_reader.py
SaaS書込	run_saas_writer	workers/micro/saas_writer.py
メッセージ生成	run_message_drafter	workers/micro/message.py
差分検出	run_diff_detector	workers/micro/diff.py
PDF生成	run_pdf_generator	workers/micro/pdf_generator.py

---

0. エグゼクティブサマリー

なぜ物流・運送BPOか

```
事業所数: 約6.3万
市場規模: 約27兆円/年
2024年問題: 残業上限規制（年960時間）→ ドライバー不足が深刻化
DX進捗: 中小は非常に遅い。配車がホワイトボード+ベテランの頭の中

運送会社は：
  ・配車計画がベテラン配車マン1人に全依存。退職=業務崩壊
  ・点呼記録・運転日報・運行指示書を手書き（法定義務）
  ・2024年問題で残業管理が生死を分ける
  ・荷主ごとにバラバラの運賃体系。請求書作成が地獄
  ・車検・点検・保険の期限管理が属人化
  → 建設業の「安全書類+施工計画+許認可」と同構造
```

支払いロジック

```
■ 直接効果①：燃料費の削減
  車両50台 × 月間燃料費20万円/台 = 月1,000万円
  ルート最適化で10%削減 = 月100万円 = 年1,200万円

■ 直接効果②：空車率の改善
  空車率40%→35%に改善
  → 実車率向上分の売上増 = 月50-100万円

■ 直接効果③：配車マンの工数削減
  配車計画: 毎朝2-3時間 × 配車マン2名 = 月120-180時間
  → AI支援で50%削減 = 月60-90時間削減
  → 時給2,000円 × 75時間 = 月15万円

■ 直接効果④：2024年問題対応
  残業上限960h/年の自動管理
  → 違反=車両停止命令。事業継続不能リスクの回避

■ 合計効果：年1,400-2,000万円（50台規模）
  → BPO月15万円（年180万円）で ROI 7.8〜11.1倍
```

8モジュール全体像

#	モジュール	Tier	月額価値	実装Phase
1	配車計画AI	★キラー	¥10-20万	Phase A ✅ 実装済み
2	運行管理	Tier1	¥3-5万	Phase B
3	車両管理	Tier2	¥2-3万	Phase B
4	傭車管理	Tier2	¥2-3万	Phase C
5	請求・運賃計算	Tier1	¥3-5万	Phase B
6	倉庫管理	Tier2	¥2-3万	Phase C
7	安全管理	Tier3	¥1-2万	Phase D
8	届出・許認可管理	Tier3	¥1-2万	Phase D

---

1. 配車計画AI（★キラーフィーチャー）

1.1 問題の本質

```
配車計画の現状:
  1. 荷主から配送依頼を受ける（電話/FAX/メール/EDI）
  2. 配車マンがドライバーの稼働状況を確認（ホワイトボード or Excel）
  3. 車両の空き・積載量・配送エリアを考慮
  4. ルートを組む（経験と勘 → 最適解ではない）
  5. ドライバーに指示（電話/LINE/点呼時に口頭）
  6. 当日の変更対応（キャンセル・追加・渋滞）→ 再配車が間に合わない

  → ベテラン配車マン1人に全依存。平均年齢50代
  → 配車マンの退職=会社の存続危機
  → 最適化されていない配車=燃料費・人件費の無駄
  → 空車率（荷物を積んでいない走行の割合）が全国平均40%
```

1.2 法令制約（ゲノムに詰める内容）

改善基準告示（自動車運転者の労働時間等の改善のための基準）

```
■ 2024年4月1日施行 改正内容

【拘束時間】
  1日の拘束時間: 原則13時間以内
    - 延長限度: 15時間以内
    - 14時間超は週2回まで
  1ヶ月の拘束時間: 原則284時間以内
    - 労使協定により310時間まで延長可（年6回まで）
  1年の拘束時間: 原則3,300時間以内
    - 労使協定により3,400時間まで延長可

【休息期間】
  継続11時間以上を基本とし、9時間を下回らないこと
  ※ 旧基準は8時間→新基準で大幅強化

【運転時間】
  1日: 2日平均で9時間以内
  1週間: 2週間平均で44時間以内

【連続運転時間】
  4時間以内に合計30分以上の休憩
  ※ 1回が概ね10分以上の休憩の合計でも可

【年間残業上限】（労働基準法改正）
  年960時間（月平均80時間）
  → 違反: 6ヶ月以下の懲役 or 30万円以下の罰金
```

法令参照:

- 労働基準法第36条（時間外労働の上限規制）
- 労働基準法第141条（自動車運転業務の特例 → 2024年4月1日施行）
- 厚生労働省告示第367号（改善基準告示 2022年12月23日改正）
- 貨物自動車運送事業法第17条（過労運転の防止）

車両制約

```
■ 車両タイプ別積載量

  | 車両種別 | 最大積載量(kg) | 車両総重量(kg) | 必要免許 |
  |---|---|---|---|
  | 軽貨物 | 350 | 3,500以下 | 普通 |
  | 小型トラック(2t) | 2,000 | 5,000〜8,000 | 普通(H19.6.2前取得)or準中型 |
  | 中型トラック(4t) | 4,000 | 8,000〜11,000 | 中型以上 |
  | 大型トラック(10t) | 10,000 | 11,000〜25,000 | 大型 |
  | トレーラー(20t) | 20,000 | 〜36,000 | 大型+けん引 |

■ 車種別特性
  | 車種 | 用途 | 特記事項 |
  |---|---|---|
  | 平ボディ | 一般貨物 | 雨天注意（シート養生必要） |
  | ウイング | パレット貨物 | フォークリフト荷役対応 |
  | 冷凍冷蔵 | 食品・医薬品 | 温度管理必須（-25℃〜+20℃） |
  | タンクローリー | 液体 | 危険物取扱者必要 |
  | ダンプ | 土砂・砕石 | 土砂等運搬大型自動車使用届出要 |
```

1.3 配車最適化アルゴリズム

```
■ Phase 1: 貪欲法 + ソート（✅ 実装済み）
  1. 配送依頼をurgent→normalでソート
  2. 各依頼に対して、積載量余裕が最大のドライバーを割当（貪欲法）
  3. ドライバー毎にtime_window.startの昇順でルートを決定
  4. 推定距離 = 配送先数 × 15km（簡易推定）
  計算量: O(n × m) （n=注文数、m=ドライバー数）

■ Phase 2: 最近傍法 + 2-opt改善（次期実装）
  1. 各ドライバーの拠点（デポ）から最も近い配送先を選択
  2. 選択した配送先から次に最も近い未訪問の配送先を選択
  3. 全配送先を訪問するまで繰り返し（最近傍法 = nearest neighbor）
  4. 2-opt改善: 任意の2辺を入れ替えて総距離が短縮すればスワップ
     - ルート内の全ペア (i, j) について:
       新距離 = dist(i-1, j) + dist(i, j+1) - dist(i-1, i) - dist(j, j+1)
       新距離 < 0 ならスワップして i〜j 間のルートを反転
  5. 改善がなくなるまで繰り返し
  改善率: 最近傍法単体比で10-15%距離短縮

■ Phase 3: OR-Tools VRP Solver（Phase 2+）
  Google OR-Tools の Vehicle Routing Problem (VRP) ソルバー
  制約: 時間枠、積載量、運転時間、休憩義務
  → 数学的に準最適解を保証
```

高速道路料金計算ロジック

```
■ 計算式（ETC2.0前提）

  基本料金 = 距離 × 距離単価
    - 普通車区間: 24.6円/km（税抜）
    - 大都市近郊区間: 29.52円/km
    - 海峡部等特別区間: 108.1円/km

  ETC割引:
    - 深夜割引（0:00-4:00）: 30%引き
    - 休日割引（土日祝）: 30%引き（普通車のみ。大型は対象外）
    - 大口・多頻度割引: 利用額に応じて最大40%引き

  車種区分:
    | 区分 | 車種 | 料金係数 |
    |---|---|---|
    | 普通車 | 軽貨物・小型トラック | 1.0 |
    | 中型車 | 中型トラック | 1.2 |
    | 大型車 | 大型トラック | 1.65 |
    | 特大車 | トレーラー | 2.75 |

  概算料金 = 基本料金 × 車種係数 × (1 - ETC割引率) × 1.1(税込)
  ※ 区間ごとの正確な料金はNEXCO API or 料金テーブルで取得（Phase 2+）
```

1.4 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_output_validator	orders[], drivers[], dispatch_date	配送依頼データ取得・検証（必須フィールド・日付チェック）	validated_orders
2	run_rule_matcher	validated_orders + drivers	ドライバー・車両マッチング（積載量/免許種別チェック→貪欲法割当）	driver_order_map
3	run_cost_calculator	driver_order_map	ルート最適化（urgent先頭→time_window昇順→推定距離・運転時間算出）	dispatch_plan
4	run_compliance_checker	dispatch_plan + driver_overtime	労働法コンプライアンスチェック（月間残業80h/休息11h/運転9h）	compliance_alerts
5	run_output_validator	dispatch_plan + compliance_alerts	バリデーション（dispatch_planの整合性検証）	DispatchPipelineResult

1.5 パイプライン詳細設計（実装）

```
# workers/bpo/logistics/pipelines/dispatch_pipeline.py（✅実装済み）

class DispatchPipeline:
    """
    配車計画AIパイプライン

    Step 1: order_reader        配送依頼データ取得・検証
      - マイクロエージェント: micro/validator
      - 入力: orders[], drivers[], dispatch_date
      - 検証: 必須フィールド存在チェック、日付フォーマットチェック
      - 出力: 検証済みオーダーリスト、ドライバーリスト

    Step 2: driver_matcher      ドライバー・車両マッチング
      - マイクロエージェント: micro/matcher
      - 処理:
        (a) urgent注文を優先ソート
        (b) 各注文に対して適合ドライバーをフィルタ
            - 積載量チェック: order.weight_kg <= VEHICLE_TYPES[driver.vehicle_type]
            - 免許種別チェック: _can_drive(driver.license_type, driver.vehicle_type)
        (c) 積載量余裕が最大のドライバーに割当（貪欲法）
        (d) 割当不可 → unmatched_ordersに追加
      - 出力: driver_order_map {driver_id: [orders]}, unmatched_orders[]

    Step 3: route_optimizer     ルート最適化（配送順序・総距離最小化）
      - マイクロエージェント: micro/route_optimizer
      - 処理:
        (a) ドライバー毎にurgent先頭、normalはtime_window.start昇順ソート
        (b) 推定距離 = 配送先数 × 15km（Phase 1 簡易推定）
        (c) 推定運転時間 = 推定距離 / 40km/h（市街地平均時速）
      - Phase 2: 最近傍法 + 2-opt、距離マトリクスAPI連携
      - 出力: dispatch_plan[], total_distance_km

    Step 4: compliance_checker  労働法コンプライアンスチェック（2024年問題）
      - マイクロエージェント: micro/compliance_checker
      - チェック項目:
        (a) 月間残業80時間超 → 超過アラート
        (b) 月間残業72時間超（90%ライン） → 警告アラート
        (c) 継続休息11時間未確保 → アラート
            last_rest_end → dispatch_date の差分 < 11h
        (d) 1日の運転予定9時間超 → アラート
      - 出力: compliance_alerts[]

    Step 5: output_validator    バリデーション
      - マイクロエージェント: micro/validator
      - 検証: dispatch_planの整合性（driver_id存在、stops配列正当性）
      - 出力: DispatchPipelineResult
    """
```

1.5 データモデル

```
// 入力: 配送依頼
{
  "orders": [
    {
      "order_id": "ORD-20260328-001",
      "shipper_id": "SHP-001",
      "shipper_name": "山田運輸株式会社",
      "origin": {
        "name": "東京倉庫",
        "address": "東京都江東区有明1-2-3",
        "lat": 35.6329,
        "lng": 139.7946
      },
      "destination": {
        "name": "横浜配送センター",
        "address": "神奈川県横浜市中区港町1-1",
        "lat": 35.4437,
        "lng": 139.6380
      },
      "weight_kg": 3500,
      "volume_m3": 12.5,
      "cargo_type": "一般貨物",
      "temperature_control": null,
      "time_window": {
        "start": "09:00",
        "end": "12:00"
      },
      "priority": "normal",
      "special_requirements": [],
      "unloading_time_min": 30
    }
  ],
  "drivers": [
    {
      "driver_id": "DRV-001",
      "name": "鈴木太郎",
      "license_type": "大型",
      "vehicle_type": "大型トラック",
      "vehicle_id": "VH-001",
      "base_location": {
        "name": "本社営業所",
        "lat": 35.6812,
        "lng": 139.7671
      },
      "monthly_overtime_hours": 65.5,
      "annual_overtime_hours": 720,
      "last_rest_end": "2026-03-27T20:00:00",
      "available": true,
      "special_qualifications": ["危険物取扱者乙4", "フォークリフト"]
```

```
    }
  ],
  "dispatch_date": "2026-03-28"
}

// 出力: 配車計画
{
  "dispatch_plan": [
    {
      "driver_id": "DRV-001",
      "driver_name": "鈴木太郎",
      "vehicle_type": "大型トラック",
      "vehicle_id": "VH-001",
      "stops": [
        {
          "order_id": "ORD-20260328-001",
          "sequence": 1,
          "origin": "東京倉庫",
          "destination": "横浜配送センター",
          "weight_kg": 3500,
          "time_window": {"start": "09:00", "end": "12:00"},
          "estimated_arrival": "10:15",
          "estimated_departure": "10:45",
          "priority": "normal"
        }
      ],
      "total_weight_kg": 3500,
      "load_ratio": 0.35,
      "estimated_distance_km": 45.0,
      "estimated_driving_hours": 1.1,
      "estimated_working_hours": 2.5,
      "toll_cost_yen": 1320,
      "fuel_cost_yen": 1800
    }
  ],
  "compliance_alerts": [],
  "total_distance_km": 45.0,
  "unmatched_orders": [],
  "summary": {
    "total_orders": 1,
    "matched_orders": 1,
    "unmatched_orders": 0,
    "total_drivers_used": 1,
    "average_load_ratio": 0.35,
    "estimated_fuel_cost_total": 1800,
    "estimated_toll_cost_total": 1320
  }
}
```

1.6 制約・前提条件

```
■ Phase 1 制約
  - GPSリアルタイム連携なし（バッチ配車のみ）
  - 距離計算は簡易推定（15km/配送先）。Google Distance Matrix API連携はPhase 2
  - 高速道路料金は概算計算。NEXCO APIはPhase 2
  - 天候・渋滞考慮なし
  - 危険物・温度管理の制約チェックは未実装

■ 前提
  - ドライバー・車両マスタが事前登録済み
  - 配送依頼は構造化データとして入力（FAX/電話のOCR入力はPhase 2）
  - 月間・年間残業時間はドライバーデータに含まれる
```

---

2. 運行管理

2.1 問題の本質

```
運行管理の現状:
  - 運行指示書は配車マンが手書き or Excel で毎日作成
  - 点呼記録は紙の点呼記録簿に記載（対面点呼の法定義務）
  - アルコールチェックは検知器の結果を手書き転記
  - 運転日報はドライバーが手書き → 事務員がExcelに転記
  → 月間100-200枚の書類を手作業で管理

法定義務:
  - 運行指示書: 貨物自動車運送事業法施行規則第9条の3
  - 点呼記録: 同第7条（乗務前・乗務後・中間点呼）
  - 運転日報: 同第8条
  - アルコールチェック: 同第7条第1項第4号（2022年4月義務化）
  - 保存期間: 1年間（運転日報・点呼記録簿）
```

2.2 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	dispatch_plan, driver_master, vehicle_master	配車計画データ読み込み	operation_data
2	run_document_generator	operation_data	運行指示書自動生成（法定記載事項充足チェック+改善基準告示注記）	instruction_pdf
3	run_document_generator	driver_data	点呼記録テンプレート生成（乗務前/乗務後/中間。アルコール検知含む）	roll_call_record
4	run_document_generator	dispatch_plan + actual_data	運転日報自動生成（法定記載事項+拘束時間・運転時間自動集計）	daily_report
5	run_rule_matcher	alcohol_check_data	アルコールチェック記録（0.15mg/L以上→即時アラート）	alcohol_record
6	run_cost_calculator	monthly_operation_data	月次集計・レポート（ドライバー別拘束時間/改善基準遵守状況）	monthly_report
7	run_output_validator	all_documents	バリデーション（法定記載事項漏れチェック）	validated_output

2.3 パイプライン詳細設計（実装）

```
# workers/bpo/logistics/pipelines/operation_management_pipeline.py

class OperationManagementPipeline:
    """
    運行管理書類パイプライン

    Step 1: dispatch_data_reader   配車計画データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: dispatch_plan（配車計画AIの出力）、driver_master、vehicle_master
      - 出力: 当日の運行データ一式

    Step 2: instruction_generator  運行指示書自動生成
      - マイクロエージェント: micro/document_generator
      - 処理:
        (a) 配車計画から運行指示書の各項目を転記
        (b) 法定記載事項の充足チェック:
            - 運行の開始・終了の地点及び日時
            - 運行の経路
            - 主な経過地における発車・到着の日時
            - 運行に際して注意を要する箇所の位置
            - 運行の途中における休憩の地点及び時間
            - 乗務員の氏名
            - 運行管理者の氏名
        (c) 改善基準告示の自動チェック結果を注記
      - 出力: 運行指示書PDF（ドライバー別）
      - 法令参照: 貨物自動車運送事業法施行規則第9条の3第2項

    Step 3: roll_call_recorder     点呼記録管理
      - マイクロエージェント: micro/form_filler
      - 処理:
        (a) 乗務前点呼の記録テンプレート生成
            - 対面点呼 or IT点呼（Gマーク事業所のみIT点呼可）
            - チェック項目:
              ・酒気帯びの有無（アルコール検知器の測定値）
              ・疾病・疲労等の状況
              ・睡眠時間
              ・日常点検の結果
              ・運行指示書の内容確認
        (b) 乗務後点呼の記録テンプレート生成
            - チェック項目:
              ・自動車・道路・運行の状況
              ・交替運転者への通告事項
              ・酒気帯びの有無
        (c) 中間点呼（2泊3日以上の運行時）
      - 出力: 点呼記録簿（日次）
      - 法令参照: 貨物自動車運送事業法施行規則第7条

    Step 4: daily_report_generator 運転日報自動生成
      - マイクロエージェント: micro/document_generator
```

```
    - 処理:
      (a) 配車計画 + 実績データ（入力 or GPS連携）から日報を生成
      (b) 法定記載事項:
          - 乗務した事業用自動車の登録番号
          - 乗務の開始・終了の地点及び日時
          - 主な経過地点及び乗務した距離
          - 運転を交替した場合の地点及び日時
          - 休憩又は仮眠をした場合の地点及び日時
          - 車両総重量8t以上又は最大積載量5t以上の場合:
            ・荷主の氏名等、荷物の積載状況
            ・集貨地点等での荷役作業の時間
      (c) 拘束時間・運転時間の自動集計
    - 出力: 運転日報（ドライバー別・日次）
    - 法令参照: 貨物自動車運送事業法施行規則第8条

  Step 5: alcohol_check_recorder アルコールチェック記録
    - マイクロエージェント: micro/form_filler
    - 処理:
      (a) アルコール検知器の測定結果を記録
      (b) 記録項目: 検査日時、ドライバー氏名、検知器の結果、確認者氏名
      (c) 0.15mg/L以上 → 即時アラート（運行中止）
      (d) 検知器の故障・未使用 → コンプライアンスアラート
    - 出力: アルコールチェック記録簿
    - 法令参照: 道路交通法施行令第44条の3（酒気帯び基準: 0.15mg/L）

  Step 6: monthly_aggregator     月次集計・レポート
    - マイクロエージェント: micro/report_generator
    - 処理:
      (a) ドライバー別月間拘束時間・運転時間の集計
      (b) 改善基準告示の遵守状況レポート
      (c) 超過傾向のドライバーに対する翌月の配車調整提案
    - 出力: 月次運行管理レポート

  Step 7: output_validator       バリデーション
    - 法定記載事項の漏れチェック
    - 改善基準告示の数値チェック
  """
```

2.3 データモデル

```
// 運行指示書
{
  "instruction_id": "INS-20260328-001",
  "dispatch_date": "2026-03-28",
  "driver_id": "DRV-001",
  "driver_name": "鈴木太郎",
  "vehicle_number": "品川100あ1234",
  "operation_manager": "田中一郎",
  "departure": {
    "location": "本社営業所",
    "datetime": "2026-03-28T07:30:00"
  },
  "return": {
    "location": "本社営業所",
    "datetime": "2026-03-28T17:00:00"
  },
  "route": [
    {
      "sequence": 1,
      "location": "東京倉庫",
      "arrival": "08:00",
      "departure": "08:30",
      "activity": "積込",
      "notes": "フォークリフト使用"
    },
    {
      "sequence": 2,
      "location": "横浜配送センター",
      "arrival": "10:15",
      "departure": "10:45",
      "activity": "荷降ろし",
      "notes": ""
    }
  ],
  "rest_breaks": [
    {
      "location": "海老名SA",
      "start": "09:30",
      "duration_min": 15,
      "reason": "連続運転2時間超"
    }
  ],
  "caution_points": [
    "東名高速 海老名JCT 工事規制あり（～4/15）",
    "横浜市中区 一方通行多し"
  ],
  "compliance_check": {
    "estimated_restraint_hours": 9.5,
    "estimated_driving_hours": 3.5,
```

```
    "rest_period_from_last_duty_hours": 14.5,
    "status": "適合"
  }
}

// 点呼記録
{
  "roll_call_id": "RC-20260328-001",
  "type": "乗務前",
  "method": "対面",
  "datetime": "2026-03-28T07:15:00",
  "driver_id": "DRV-001",
  "driver_name": "鈴木太郎",
  "checker_name": "田中一郎",
  "alcohol_check": {
    "result_mg_per_l": 0.00,
    "device_id": "ALC-001",
    "status": "正常"
  },
  "health_status": {
    "fatigue": "なし",
    "illness": "なし",
    "sleep_hours": 7.5
  },
  "daily_inspection": "完了・異常なし",
  "instruction_confirmed": true,
  "notes": ""
}
```

---

3. 車両管理

3.1 問題の本質

```
車両管理の現状:
  - 車検・定期点検の期限管理がExcelかカレンダーの手書き
  - 期限切れで車両停止 → 配車に支障
  - 車両ごとのコスト（燃料/メンテ/保険/リース料）が把握できていない
  - 稼働率が不明（使われていない車両にリース料を払い続ける）

  50台規模の場合:
    車両関連コスト = 月間約500-800万円
    適切な管理で5-10%削減 = 月25-80万円の効果
```

3.2 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	vehicle_master, maintenance_records, fuel_records	車両マスタ・実績データ読み込み	vehicle_data
2	run_rule_matcher	vehicle_data	期限管理（車検/3ヶ月定期点検/自賠責/任意保険。90日/30日/7日前アラート）	deadline_alerts
3	run_cost_calculator	vehicle_data + fuel_records	車両別コスト分析（燃料費+メンテ+保険+リース+税金→km単価算出）	cost_analysis
4	run_cost_calculator	vehicle_data + dispatch_records	稼働率分析（稼働率/実車率/積載効率。60%未満→廃車検討）	utilization_analysis
5	run_document_generator	cost_analysis + utilization_analysis	車両管理レポート生成（車両別コスト/稼働率/期限一覧）	vehicle_report
6	run_output_validator	vehicle_report	バリデーション	validated_output

3.3 パイプライン詳細設計（実装）

```
# workers/bpo/logistics/pipelines/vehicle_management_pipeline.py

class VehicleManagementPipeline:
    """
    車両管理パイプライン

    Step 1: vehicle_data_reader      車両マスタ・実績データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: vehicle_master、maintenance_records、fuel_records
      - 出力: 車両一覧 + 各種期限情報

    Step 2: deadline_checker         期限管理（車検・点検・保険）
      - マイクロエージェント: micro/deadline_monitor
      - 処理:
        (a) 車検有効期限チェック
            - 新車: 初回3年、以降2年（事業用は1年）
            - 事業用貨物自動車: 初回2年、以降1年
            - 法令参照: 道路運送車両法第61条
        (b) 3ヶ月定期点検チェック（事業用は3ヶ月ごと法定義務）
            - 法令参照: 道路運送車両法第48条第1項
        (c) 自賠責保険・任意保険有効期限チェック
        (d) アラート生成:
            - 90日前: 準備開始通知
            - 30日前: 要対応通知
            - 7日前: 緊急通知
      - 出力: deadline_alerts[]

    Step 3: cost_analyzer            車両別コスト分析
      - マイクロエージェント: micro/cost_calculator
      - 計算式:
        月間車両コスト = 燃料費 + メンテナンス費 + 保険料 + リース料 + 税金(月割)

        燃料費 = 走行距離(km) / 燃費(km/L) × 軽油単価(円/L)
          - 大型トラック燃費: 3.0-4.5 km/L
          - 中型トラック燃費: 5.0-7.0 km/L
          - 小型トラック燃費: 8.0-12.0 km/L
          - 軽貨物燃費: 14.0-18.0 km/L
          - 軽油単価: 全国平均 約155円/L（2026年3月時点）

        km単価 = 月間車両コスト / 月間走行距離
        → 車両ごとのkm単価を比較して非効率車両を特定

    Step 4: utilization_analyzer     稼働率分析
      - マイクロエージェント: micro/analyzer
      - 計算式:
        稼働率 = 実稼働日数 / 営業日数 × 100(%)
        実車率 = 実車距離 / 総走行距離 × 100(%)（空車率 = 100% - 実車率）
        積載効率 = 実積載量 / 最大積載量 × 100(%)
```

```
        判定基準:
          稼働率 80%以上: 良好
          稼働率 60-80%: 要改善（代替検討）
          稼働率 60%未満: 廃車・売却検討

    Step 5: report_generator         車両管理レポート生成
      - マイクロエージェント: micro/report_generator
      - 出力: 車両別コストレポート、稼働率レポート、期限一覧

    Step 6: output_validator         バリデーション
    """
```

3.3 データモデル

```
// 車両マスタ
{
  "vehicle_id": "VH-001",
  "company_id": "COM-001",
  "registration_number": "品川100あ1234",
  "vehicle_type": "大型トラック",
  "body_type": "ウイング",
  "max_payload_kg": 10000,
  "gross_weight_kg": 24500,
  "manufacturer": "日野",
  "model": "プロフィア",
  "year": 2022,
  "fuel_type": "軽油",
  "ownership": "リース",
  "lease_monthly_yen": 280000,
  "insurance": {
    "compulsory_expiry": "2027-03-15",
    "voluntary_expiry": "2027-03-15",
    "voluntary_premium_annual": 350000
  },
  "inspection": {
    "last_inspection_date": "2025-10-01",
    "next_inspection_due": "2026-10-01",
    "periodic_3month_due": "2026-04-01"
  },
  "annual_tax_yen": 40500,
  "status": "稼働中"
}
```

4. 傭車管理（外部委託車両）

4.1 問題の本質

```
傭車の現状:
  - 自社車両で対応しきれない案件を外部の運送会社に委託
  - 中小運送会社の傭車比率: 平均20-40%
  - 傭車先とのやり取りは電話/FAX。依頼書も手書き
  - 下請法の60日支払ルールの管理が曖昧
  → 支払遅延で下請法違反のリスク
```

4.2 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	unmatched_orders + charter_master	傭車依頼データ読み込み（配車計画の未割当案件+傭車先マスタ）	charter_request
2	run_rule_matcher	charter_request	傭車先マッチング（対応エリア/車両タイプ/実績スコアリング/運賃妥当性）	recommended_charters
3	run_document_generator	recommended_charters	傭車依頼書自動生成（運送委託契約書+標準運賃比較表示）	charter_document
4	run_compliance_checker	charter_document	下請法準拠チェック（書面交付/60日支払/買いたたき禁止）	compliance_result
5	run_cost_calculator	compliance_result	支払予定管理（傭車先別支払予定+60日ルール期限自動計算+7日前アラート）	payment_schedule
6	run_output_validator	payment_schedule	バリデーション	validated_output

4.3 パイプライン詳細設計（実装）

```
# workers/bpo/logistics/pipelines/charter_management_pipeline.py

class CharterManagementPipeline:
    """
    傭車管理パイプライン

    Step 1: charter_request_reader   傭車依頼データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 配車計画のunmatched_orders + 傭車先マスタ
      - 出力: 傭車が必要な案件リスト + 候補傭車先リスト

    Step 2: charter_matcher          傭車先マッチング
      - マイクロエージェント: micro/matcher
      - 処理:
        (a) 傭車先の対応エリア・車両タイプ・実績でフィルタ
        (b) 過去の評価（品質/定時率/単価）でスコアリング
        (c) 運賃の妥当性チェック（市場相場との比較）
      - 出力: 推奨傭車先リスト（スコア付き）

    Step 3: document_generator       傭車依頼書自動生成
      - マイクロエージェント: micro/document_generator
      - 処理:
        (a) 傭車依頼書（運送委託契約書）の自動生成
            - 記載事項: 荷主名、積地・降地、荷物情報、運賃、支払条件
        (b) 標準的な運賃（2024年改正告示）との比較表示
      - 出力: 傭車依頼書PDF

    Step 4: compliance_checker       下請法準拠チェック
      - マイクロエージェント: micro/compliance_checker
      - チェック項目:
        (a) 書面交付義務（発注時に書面を交付しているか）
            - 下請代金支払遅延等防止法第3条
        (b) 60日以内支払ルール
            - 下請代金支払遅延等防止法第2条の2
            - 受領日から起算して60日以内に支払うこと
            - 支払期日の自動計算: 納品日 + 60日（60日超 → アラート）
        (c) 買いたたきの禁止
            - 標準的な運賃（国土交通省告示第424号 2024年改正）との比較
            - 市場相場から20%以上乖離 → 警告
        (d) 不当な返品・やり直し要求の禁止
      - 法令参照:
        - 下請代金支払遅延等防止法（下請法）第2条の2、第3条、第4条
        - 国土交通省「標準的な運賃」告示（2024年3月改正）
      - 出力: compliance_alerts[]

    Step 5: payment_scheduler        支払予定管理
      - マイクロエージェント: micro/payment_manager
      - 処理:
        (a) 傭車先別の支払予定一覧生成
```

```
        (b) 60日ルールに基づく支払期限の自動計算
        (c) 支払期限7日前アラート
      - 出力: payment_schedule[]

    Step 6: output_validator         バリデーション
    """
```

---

5. 請求・運賃計算

5.1 問題の本質

```
運賃計算の現状:
  - 荷主ごとにバラバラの運賃体系（距離制/重量制/個建て/貸切）
  - 燃料サーチャージの計算が手作業
  - 月末の請求書作成がExcel地獄（1荷主あたり2-4時間 × 荷主数）
  - 請求漏れ・計算ミスで年間数十万〜数百万の損失
```

5.2 運賃計算式

```
■ 距離制運賃（トラック貸切）

  基本運賃 = 距離制基本運賃表[車両区分][距離帯]
  ※ 国土交通省「標準的な運賃」告示（2024年3月改正）に基づく

  距離帯の例（大型車・関東）:
    | 距離帯(km) | 運賃(円/車) |
    |---|---|
    | ～20 | 19,810 |
    | ～40 | 24,690 |
    | ～60 | 29,240 |
    | ～80 | 33,780 |
    | ～100 | 37,660 |
    | ～150 | 46,150 |
    | ～200 | 55,020 |
    | 200超 | 55,020 + (距離-200) × 217 |

■ 重量制運賃（特積み・路線便）

  基本運賃 = 重量制基本運賃表[重量帯][距離帯]

  例（関東・100km）:
    | 重量帯(kg) | 運賃(円) |
    |---|---|
    | ～100 | 2,700 |
    | ～500 | 5,400 |
    | ～1,000 | 8,100 |
    | ～2,000 | 13,500 |
    | ～5,000 | 27,000 |

■ 個建て運賃（宅配・小口）

  基本運賃 = 個建て単価[サイズ][距離帯] × 個数

■ 燃料サーチャージ計算

  サーチャージ額 = 基本運賃 × サーチャージ率

  サーチャージ率 = (当月軽油価格 - 基準価格) / 基準価格 × 100(%)
    - 基準価格: 120円/L（一般的な設定）
    - 例: 軽油155円の場合 → (155-120)/120 = 29.2% → 端数処理で29%

  適用条件: 荷主との契約に燃料サーチャージ条項がある場合
  ※ 標準的な運賃告示では燃料サーチャージの収受を推奨

■ 割増料金

  | 項目 | 割増率 |
  |---|---|
```

```
    | 休日割増 | +20% |
    | 深夜・早朝割増（22:00-5:00） | +30% |
    | 冷凍・冷蔵 | +20-30% |
    | 危険物 | +30-50% |
    | 待機時間（30分超過分） | 時間あたり加算 |

■ 請求額計算式（最終）

  請求額 = (基本運賃 + 付帯作業料) × (1 + 割増率) × (1 + サーチャージ率)
           + 高速道路料金（実費）
           + 待機時間料金
           → 消費税10%加算
```

5.3 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	delivery_records, shipper_master, fare_tables	配送実績データ読み込み（荷主別締日設定対応）	delivery_data
2	run_cost_calculator	delivery_data + fare_tables	運賃計算（距離制/重量制/個建て+割増+燃料サーチャージ+高速+待機）	fare_result
3	run_diff_detector	fare_result + historical_data	異常検知（前月比±30%超/同一ルート過去実績比較/標準運賃告示比較）	anomaly_alerts
4	run_document_generator	fare_result	請求書自動生成（インボイス制度対応+電子帳簿保存法対応）	invoice_pdf
5	run_rule_matcher	invoice_pdf + bank_data	売掛管理（請求残高更新+入金消込+支払期日超過アラート）	receivable_report
6	run_output_validator	receivable_report	バリデーション	validated_output

5.4 パイプライン詳細設計（実装）

```
# workers/bpo/logistics/pipelines/billing_pipeline.py

class BillingPipeline:
    """
    請求・運賃計算パイプライン

    Step 1: delivery_data_reader      配送実績データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 配送実績、荷主マスタ、運賃テーブル
      - 締日: 荷主ごとの締日設定（末締め/20日締め/15日締め等）

    Step 2: fare_calculator           運賃計算
      - マイクロエージェント: micro/fare_calculator
      - 処理:
        (a) 荷主別の運賃体系を適用（距離制/重量制/個建て/固定）
        (b) 割増料金の適用（休日/深夜/冷凍/危険物）
        (c) 燃料サーチャージの自動計算
        (d) 高速道路料金の加算
        (e) 待機時間料金の加算（30分超過分、30分単位で加算）
      - 出力: 明細付き運賃計算結果

    Step 3: anomaly_detector          異常検知
      - マイクロエージェント: micro/anomaly_detector
      - 処理:
        (a) 前月比の大幅変動検知（±30%以上）
        (b) 同一荷主・同一ルートの過去実績との比較
        (c) 単価の妥当性チェック（標準的な運賃告示との比較）
      - 出力: anomaly_alerts[]

    Step 4: invoice_generator         請求書自動生成
      - マイクロエージェント: micro/document_generator
      - 処理:
        (a) 荷主別・締日別に請求書を生成
        (b) インボイス制度対応（適格請求書の記載事項）
            - 適格請求書発行事業者の登録番号（T+13桁）
            - 税率ごとの消費税額と合計額
        (c) 電子帳簿保存法対応（タイムスタンプ）
      - 出力: 請求書PDF

    Step 5: receivable_manager        売掛管理
      - マイクロエージェント: micro/receivable_manager
      - 処理:
        (a) 請求残高一覧の更新
        (b) 入金消込（銀行API or CSV取込）
        (c) 支払期日超過アラート（30日/60日/90日超）
      - 出力: 売掛金残高レポート
```

```
    Step 6: output_validator         バリデーション
    """
```

---

6. 倉庫管理

6.1 問題の本質

```
倉庫管理の現状（運送会社の自社倉庫）:
  - 入出庫記録が紙の台帳 or 簡易Excel
  - ロケーション管理なし（「あのへんにあるはず」）
  - 棚卸しが年1-2回で差異が大きい
  - 保管料の計算が手作業

対象: 運送会社が荷主の荷物を預かる営業倉庫（3PL含む）
```

6.2 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	io_slips (CSV/手入力), inventory_master	入出庫データ読み込み→在庫更新	updated_inventory
2	run_rule_matcher	updated_inventory	ロケーション管理（ABC分析→出荷頻度順配置提案）	location_proposal
3	run_diff_detector	inventory_master + actual_count	棚卸支援（循環棚卸A=毎月/B=四半期/C=年1。差異率5%超→原因調査アラート）	stocktaking_report
4	run_cost_calculator	inventory_data + rate_table	保管料計算（坪建て/パレット建て/個建て+荷役料）	storage_fee
5	run_document_generator	updated_inventory + storage_fee	在庫レポート生成（在庫一覧/入出庫履歴/保管料明細/稼働率）	inventory_report
6	run_output_validator	inventory_report	バリデーション	validated_output

6.3 パイプライン詳細設計（実装）

```
# workers/bpo/logistics/pipelines/warehouse_pipeline.py

class WarehousePipeline:
    """
    倉庫管理パイプライン

    Step 1: inventory_data_reader    入出庫データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 入出庫伝票（CSV/手入力）、在庫マスタ
      - 出力: 更新された在庫データ

    Step 2: location_manager         ロケーション管理
      - マイクロエージェント: micro/location_optimizer
      - 処理:
        (a) ABC分析に基づくロケーション提案
            A品目（出荷頻度上位20%）→ 出荷口近くの1階・取りやすい高さ
            B品目（中間60%）→ 中間エリア
            C品目（下位20%）→ 奥・上段
        (b) ロケーション番号体系: 棟-階-列-段（例: A-1-03-02）
      - 出力: ロケーション配置提案

    Step 3: stocktaking_support      棚卸支援
      - マイクロエージェント: micro/stocktaking
      - 処理:
        (a) 循環棚卸: A品目=毎月、B品目=四半期、C品目=年1回
        (b) 棚卸差異分析
            差異率 = |帳簿数量 - 実数量| / 帳簿数量 × 100(%)
            差異率 5%超 → 原因調査アラート
        (c) 棚卸結果レポート生成
      - 出力: 棚卸差異レポート

    Step 4: storage_fee_calculator   保管料計算
      - マイクロエージェント: micro/fee_calculator
      - 計算式:
        ■ 坪建て保管料
          保管料 = 使用坪数 × 坪単価(円/坪/月)
          坪単価目安: 都心5,000-10,000円、郊外2,000-5,000円

        ■ パレット建て保管料
          保管料 = パレット数 × 単価(円/パレット/日) × 日数
          単価目安: 100-300円/パレット/日

        ■ 個建て保管料
          保管料 = 保管個数 × 単価(円/個/日) × 日数

        ■ 荷役料（入出庫時）
          入庫料 = 数量 × 入庫単価
          出庫料 = 数量 × 出庫単価
          仕分け料 = 数量 × 仕分け単価
```

```
      - 出力: 荷主別保管料明細

    Step 5: report_generator          在庫レポート生成
      - マイクロエージェント: micro/report_generator
      - 出力: 在庫一覧、入出庫履歴、保管料明細、稼働率レポート

    Step 6: output_validator          バリデーション
    """
```

---

7. 安全管理

7.1 問題の本質

```
安全管理の現状:
  - 事故・ヒヤリハット報告が紙ベースで分析不能
  - 安全教育の記録管理が属人化
  - Gマーク（安全性優良事業所）の取得・維持に膨大な書類作業

Gマーク:
  - 全日本トラック協会が認定する安全性優良事業所制度
  - 取得率: 約30%（大手は大半取得、中小は低い）
  - メリット: 保険料割引（最大3%）、荷主からの信頼向上、入札加点
```

7.2 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_structured_extractor	incident_reports, near_miss_reports, dashcam_data	事故・ヒヤリハット記録の構造化	structured_incidents
2	run_structured_extractor	structured_incidents	事故・ヒヤリハット分析（種別分類/ハインリッヒ法則/パターン分析/4M分析）	incident_analysis
3	run_rule_matcher	driver_training_records	安全教育記録管理（初任/一般12項目/事故惹起者/適性診断の期限管理）	training_status
4	run_document_generator	company_data + incident_analysis	Gマーク申請支援（評価項目自己チェック/不足項目改善提案/申請書類生成）	gmark_assessment
5	run_document_generator	incident_analysis + training_status	安全管理レポート生成（月次安全レポート+教育実施状況）	safety_report
6	run_output_validator	safety_report	バリデーション	validated_output

7.3 パイプライン詳細設計（実装）

```
# workers/bpo/logistics/pipelines/safety_management_pipeline.py

class SafetyManagementPipeline:
    """
    安全管理パイプライン

    Step 1: incident_data_reader     事故・ヒヤリハット記録読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 事故報告書、ヒヤリハット報告、ドラレコ分析結果
      - 出力: 構造化されたインシデントデータ

    Step 2: incident_analyzer        事故・ヒヤリハット分析
      - マイクロエージェント: micro/analyzer（LLM活用）
      - 処理:
        (a) 事故種別分類（追突/脱輪/荷崩れ/接触/対人/対物）
        (b) ハインリッヒの法則に基づく分析
            1件の重大事故 : 29件の軽微事故 : 300件のヒヤリハット
        (c) 発生パターン分析（時間帯/天候/場所/ドライバー別）
        (d) 根本原因分析（4M分析: Man/Machine/Media/Management）
      - 出力: インシデント分析レポート

    Step 3: training_manager         安全教育記録管理
      - マイクロエージェント: micro/training_manager
      - 処理:
        (a) 初任運転者教育（15時間以上 + 実車20時間以上）
            - 対象: 新たに雇い入れた運転者
            - 法令参照: 貨物自動車運送事業輸送安全規則第10条第2項
        (b) 一般教育（年間12項目の指導 × 全ドライバー）
            - 法令参照: 同規則第10条第1項
            - 12項目: トラックの構造上の特性/安全運行のために遵守すべき基本的事項 他
        (c) 事故惹起者教育（事故後の特別指導12時間以上 + 適性診断受診）
            - 法令参照: 同規則第10条第2項
        (d) 適性診断（初任/適齢/特定）の受診期限管理
            - 初任診断: 雇入れ後1年以内
            - 適齢診断: 65歳以上は3年ごと
            - 特定診断: 事故惹起者は事故後1年以内
      - 出力: 教育記録一覧、受講予定アラート

    Step 4: g_mark_support           Gマーク申請支援
      - マイクロエージェント: micro/document_generator（LLM活用）
      - 処理:
        (a) Gマーク評価項目の自己チェック
            - 安全性に対する法令の遵守状況（40点）
            - 事故や違反の状況（40点）
            - 安全性に対する取組の積極性（20点）
            - 合格基準: 80点以上（初回）/ 70点以上（更新）
        (b) 不足項目の改善提案
        (c) 申請書類の自動生成支援
      - 出力: Gマーク自己評価シート、改善提案レポート
```

```
    Step 5: report_generator          安全管理レポート生成
      - マイクロエージェント: micro/report_generator
      - 出力: 月次安全管理レポート、教育実施状況レポート

    Step 6: output_validator          バリデーション
    """
```

7.3 データモデル

```
// 事故・ヒヤリハット記録
{
  "incident_id": "INC-20260328-001",
  "type": "ヒヤリハット",
  "severity": "軽微",
  "datetime": "2026-03-28T14:30:00",
  "driver_id": "DRV-003",
  "driver_name": "佐藤次郎",
  "vehicle_id": "VH-005",
  "location": {
    "address": "東京都品川区東品川2丁目",
    "lat": 35.6186,
    "lng": 139.7489
  },
  "incident_type": "接触未遂",
  "description": "左折時に歩行者を見落としそうになった",
  "cause_4m": {
    "man": "左側の死角確認不足",
    "machine": "左サイドミラーの汚れ",
    "media": "交差点手前の電柱で視界遮蔽",
    "management": "当該箇所の注意喚起未実施"
  },
  "countermeasure": "左折時の指差し確認を徹底。当該箇所を注意箇所リストに追加",
  "dashcam_file": "DRC-20260328-143000.mp4"
}

// 安全教育記録
{
  "training_id": "TRN-20260315-001",
  "type": "一般教育",
  "topic": "トラックの構造上の特性",
  "date": "2026-03-15",
  "duration_hours": 1.5,
  "instructor": "田中一郎（運行管理者）",
  "attendees": [
    {"driver_id": "DRV-001", "name": "鈴木太郎", "attended": true},
    {"driver_id": "DRV-002", "name": "高橋花子", "attended": true},
    {"driver_id": "DRV-003", "name": "佐藤次郎", "attended": false, "reason": "乗務中"}
  ],
  "materials": ["安全教育テキスト第1章", "社内事故事例集"],
  "notes": "佐藤は次回補講予定（3/22）"
}
```

8. 届出・許認可管理

8.1 問題の本質

```
届出・許認可の現状:
  - 一般貨物自動車運送事業の認可を維持するための各種届出が多数
  - 期限管理が属人化（行政書士に丸投げ or 経営者が覚えている）
  - 届出漏れ → 行政処分（車両停止/事業停止）のリスク

主な届出義務:
  1. 事業報告書: 毎事業年度経過後100日以内（7月10日前後）
  2. 事業実績報告書: 毎年7月10日まで
  3. 認可変更届出: 事業計画変更の都度
  4. 運行管理者選任届: 選任後15日以内
  5. 整備管理者選任届: 選任後15日以内
```

8.2 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	company_master, permit_list, filing_history	許認可・届出マスタ読み込み	permit_status
2	run_rule_matcher	permit_status	届出期限管理（定期届出+変更届出要否チェック。90日/30日/7日前アラート）	deadline_alerts
3	run_document_generator	company_data + delivery_records + accounting_data	事業報告書・実績報告書の自動生成（輸送実績+車両数+営業収入+事故状況）	business_reports
4	run_document_generator	change_event_data	認可変更届出管理（車両台数変更/営業所変更/役員変更の届出書類自動生成）	change_notifications
5	run_document_generator	permit_status + compliance_data	コンプライアンスダッシュボード（全許認可状況一覧+行政処分リスク評価）	compliance_dashboard
6	run_output_validator	all_filings	バリデーション	validated_output

8.3 パイプライン詳細設計（実装）

```
# workers/bpo/logistics/pipelines/permit_filing_pipeline.py

class PermitFilingPipeline:
    """
    届出・許認可管理パイプライン

    Step 1: permit_data_reader       許認可・届出マスタ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 事業者マスタ、許認可一覧、届出履歴
      - 出力: 現在の許認可状況 + 届出スケジュール

    Step 2: deadline_monitor         届出期限管理
      - マイクロエージェント: micro/deadline_monitor
      - 処理:
        (a) 定期届出の期限チェック
            - 事業報告書: 毎事業年度終了後100日以内
              法令参照: 貨物自動車運送事業報告規則第2条
            - 事業実績報告書: 毎年4/1〜翌3/31の実績を7/10まで
              法令参照: 同規則第2条の2
        (b) 変更届出の要否チェック
            - 営業所の新設・廃止・移転 → 認可申請
            - 車両数の変更 → 届出（増車30%以内は届出、超は認可）
            - 役員変更 → 届出（変更後30日以内）
              法令参照: 貨物自動車運送事業法第9条、第16条
        (c) アラート生成
            - 90日前: 準備開始
            - 30日前: 書類作成着手
            - 7日前: 最終確認
      - 出力: deadline_alerts[]

    Step 3: report_generator         事業報告書・実績報告書の自動生成
      - マイクロエージェント: micro/document_generator（LLM活用）
      - 処理:
        (a) 事業報告書の自動生成
            - 事業概況報告書（一般貨物自動車運送事業）
            - 損益明細表
            - 人件費明細表
            - 営業所別の輸送実績
            ※ 会計データ + 配送実績データから自動集計
        (b) 事業実績報告書の自動生成
            - 事業用自動車等の数（車種別）
            - 走行キロ（延実在車両数 × 1車平均走行キロ）
            - 輸送トン数
            - 営業収入
            - 交通事故の状況
            ※ 配送実績 + 車両管理データから自動集計
      - 出力: 事業報告書・実績報告書の下書き（Excel/PDF）

    Step 4: change_notification      認可変更届出管理
```

```
    - マイクロエージェント: micro/document_generator
    - 処理:
      (a) 車両台数変更届出書の自動生成
          - 増車の場合: 法令試験合格が前提（運行管理者）
          - 減車の場合: 最低車両数（5台）を下回らないこと
            法令参照: 貨物自動車運送事業法施行規則第2条
      (b) 営業所変更の認可申請書類
      (c) 役員変更届出書
    - 出力: 各種届出書類の下書き

  Step 5: compliance_dashboard      コンプライアンスダッシュボード
    - マイクロエージェント: micro/report_generator
    - 処理: 全許認可・届出の状況一覧、行政処分リスク評価
    - 出力: 許認可管理ダッシュボードデータ

  Step 6: output_validator          バリデーション
  """
```

8.3 データモデル

```
// 許認可マスタ
{
  "permit_id": "PRM-001",
  "company_id": "COM-001",
  "permit_type": "一般貨物自動車運送事業",
  "permit_number": "関自貨第1234号",
  "issue_date": "2020-04-01",
  "issuing_authority": "関東運輸局",
  "offices": [
    {
      "office_name": "本社営業所",
      "address": "東京都品川区東品川2-3-4",
      "vehicle_count": 50,
      "operation_managers": [
        {"name": "田中一郎", "qualification_number": "UM-12345", "type": "運行管理者"}
      ],
      "maintenance_managers": [
        {"name": "山本三郎", "qualification_number": "MM-67890", "type": "整備管理者"}
      ]
    }
  ],
  "filings": [
    {
      "filing_type": "事業報告書",
      "due_date": "2026-07-10",
      "status": "未着手",
      "last_filed": "2025-07-08"
    },
    {
      "filing_type": "事業実績報告書",
      "due_date": "2026-07-10",
      "status": "未着手",
      "last_filed": "2025-07-09"
    }
  ]
}
```

競合環境

サービス	月額	強み	弱み
LYNA（ライナロジクス）	要問合せ	AI配車最適化の先駆者	大手向け。中小には高額
Cariot（フレクト）	¥3-10万	動態管理+配車支援	配車最適化は限定的
MOVO（Hacobu）	要問合せ	物流DXプラットフォーム	荷主向け。運送会社向けは弱い
ロジスティクス・プロ	¥5-15万	運行管理+労務管理	AI配車なし
Loogia（オプティマインド）	要問合せ	ラストワンマイル配車	幹線輸送には不向き

シャチョツーの差別化:
- 既存AI配車は大手向け（100台+）で高額。シャチョツーは10-50台の中小運送に特化
- 配車単機能ではなく、配車+運行管理+労務+運賃+車両管理を一気通貫
- 建設業等の他業界BPOとの横連携（荷主が建設会社の場合の自動連携）
- ゲノム駆動で業種固有のルール（改善基準告示等）を構造化

DB設計（追加テーブル）

```
-- 配送依頼
CREATE TABLE dispatch_orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  order_number TEXT NOT NULL,
  shipper_id UUID,                      -- 荷主ID
  shipper_name TEXT NOT NULL,
  origin_address TEXT NOT NULL,
  origin_lat DECIMAL(9,6),
  origin_lng DECIMAL(9,6),
  destination_address TEXT NOT NULL,
  destination_lat DECIMAL(9,6),
  destination_lng DECIMAL(9,6),
  weight_kg DECIMAL(10,2),
  volume_m3 DECIMAL(10,2),
  cargo_type TEXT DEFAULT '一般貨物',
  temperature_requirement TEXT,         -- 冷凍(-25〜-18)/冷蔵(0〜10)/常温
  time_window_start TIME,
  time_window_end TIME,
  priority TEXT DEFAULT 'normal',       -- urgent/normal
  dispatch_date DATE NOT NULL,
  status TEXT DEFAULT 'pending',        --
pending/assigned/in_transit/delivered/cancelled
  assigned_driver_id UUID,
  assigned_vehicle_id UUID,
  fare_yen INTEGER,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- ドライバーマスタ
CREATE TABLE drivers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name TEXT NOT NULL,
  license_type TEXT NOT NULL,           -- 普通/準中型/中型/大型/大型特殊/けん引
  license_expiry DATE,
  vehicle_id UUID,                      -- 主に使用する車両
  base_office TEXT,                     -- 所属営業所
  monthly_overtime_hours DECIMAL(5,1) DEFAULT 0,
  annual_overtime_hours DECIMAL(6,1) DEFAULT 0,
  employment_type TEXT DEFAULT '正社員', -- 正社員/契約社員/アルバイト
  hire_date DATE,
  qualifications JSONB DEFAULT '[]',    -- ["危険物取扱者乙4", "フォークリフト"]
  status TEXT DEFAULT 'active',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
```

```
);

-- 車両マスタ
CREATE TABLE vehicles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  registration_number TEXT NOT NULL,     -- ナンバープレート
  vehicle_type TEXT NOT NULL,            -- 軽貨物/小型トラック/中型トラック/大型トラック/トレー
ラー
  body_type TEXT,                        -- 平ボディ/ウイング/冷凍冷蔵/タンク/ダンプ
  max_payload_kg INTEGER NOT NULL,
  gross_weight_kg INTEGER,
  manufacturer TEXT,
  model TEXT,
  year INTEGER,
  fuel_type TEXT DEFAULT '軽油',
  fuel_efficiency_km_per_l DECIMAL(4,1),
  ownership TEXT DEFAULT '自社',         -- 自社/リース
  lease_monthly_yen INTEGER,
  inspection_due DATE,                   -- 車検有効期限
  periodic_3month_due DATE,              -- 3ヶ月点検期限
  insurance_compulsory_expiry DATE,      -- 自賠責有効期限
  insurance_voluntary_expiry DATE,       -- 任意保険有効期限
  status TEXT DEFAULT '稼働中',          -- 稼働中/整備中/休車/廃車
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 運賃テーブル（荷主別）
CREATE TABLE fare_tables (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  shipper_id UUID,
  shipper_name TEXT NOT NULL,
  fare_type TEXT NOT NULL,               -- distance/weight/piece/fixed
  vehicle_class TEXT,                    -- 大型/中型/小型/軽
  distance_brackets JSONB,               -- [{"max_km": 20, "fare_yen": 19810}, ...]
  weight_brackets JSONB,                 -- [{"max_kg": 100, "fare_yen": 2700}, ...]
  surcharge_base_price_yen DECIMAL(6,1) DEFAULT 120, -- 燃料サーチャージ基準価格
  surcharge_enabled BOOLEAN DEFAULT true,
  holiday_surcharge_rate DECIMAL(3,2) DEFAULT 0.20,
  night_surcharge_rate DECIMAL(3,2) DEFAULT 0.30,
  effective_from DATE NOT NULL,
  effective_to DATE,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE dispatch_orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE drivers ENABLE ROW LEVEL SECURITY;
ALTER TABLE vehicles ENABLE ROW LEVEL SECURITY;
ALTER TABLE fare_tables ENABLE ROW LEVEL SECURITY;
```

```
-- ==========================================
-- 物流BPO テナント分離RLSポリシー
-- 全テーブル共通: company_id ベースのRLS
-- ==========================================

CREATE POLICY dispatch_orders_tenant ON dispatch_orders
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY drivers_tenant ON drivers
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY vehicles_tenant ON vehicles
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY fare_tables_tenant ON fare_tables
    USING (company_id = current_setting('app.company_id', true)::UUID);
```

実装ロードマップ

```
Phase A（Week 1-2）: ★最優先
  ✅ #1 配車計画AI（dispatch_pipeline.py 実装済み）
  → #2 運行管理（運行指示書・点呼記録・運転日報の自動生成）
  → #5 請求・運賃計算（荷主別運賃テーブル + 請求書自動生成）

Phase B（Week 3）:
  → #3 車両管理（期限管理 + コスト分析）
  → #4 傭車管理（下請法準拠 + 支払管理）

Phase C（Week 4）:
  → #6 倉庫管理（入出庫 + 棚卸 + 保管料計算）

Phase D（Week 5）:
  → #7 安全管理（事故分析 + 教育記録 + Gマーク支援）
  → #8 届出・許認可管理（事業報告書 + 届出期限管理）
```